

Advanced Algorithms and Data Structures

Lecture# 04: Sorting Algorithms

Types of Sorting Algorithms

- Non-recursive/Incremental comparison sorting
 - Selection sort
 - Bubble sort
 - Insertion sort
- Recursive comparison sorting
 - Merge sort
 - Quick sort
 - Heap sort
- Non-comparison linear sorting
 - Count sort
 - Radix sort
 - Bucket sort

Selection Sort

- Idea:
 - Find the smallest element in the array
 - Exchange it with the element in the first position
 - Find the second smallest element and exchange it with the element in the second position
 - Continue until the array is sorted

8	4	6	9	2	3	1
---	---	---	---	---	---	---

1	4	6	9	2	3	8
---	---	---	---	---	---	---

1	2	6	9	4	3	8
---	---	---	---	---	---	---

1	2	3	9	4	6	8
---	---	---	---	---	---	---

1	2	3	4	9	6	8
---	---	---	---	---	---	---

1	2	3	4	6	9	8
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

Selection Sort Analysis

Alg.: SELECTION-SORT(*A*)

$n \leftarrow \text{length}[A]$

for $j \leftarrow 0$ to $n - 2$

$\text{smallest} \leftarrow j$

 for $i \leftarrow j + 1$ to $n-1$

 if $A[i] < A[\text{smallest}]$

$\text{smallest} \leftarrow i$

 exchange $A[j]$ $A[\text{smallest}]$

Cost Steps

c_1 1

c_2 $n-1$

c_3 $n-1$

c_4 $\sum_{j=0}^{n-2} t_j$

c_5 $\sum_{j=0}^{n-2} t_j$

c_6 $\sum_{j=0}^{n-2} t_j$

c_7 $n-1$

t_j : # of times the inner for loop statement is executed at iteration j

$$T(n) = c_1 + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7(n-1)$$

Best/Worst/Average Case Analysis

$$T(n) = c_1 + c_2(n-1) + c_3(n-1) + c_4 \frac{n(n-1)}{2} + c_5 \frac{n(n-1)}{2} + c_6 \frac{n(n-1)}{2} + c_7(n-1)$$

$$= an^2 + bn + c$$

$$T(n) = \Theta(n^2)$$

$$\sum_{j=0}^{n-2} j = \frac{n(n-1)}{2}$$

Disadvantage:

$\Theta(n^2)$ running time in **all** cases

$$T(n) = c_1 + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7(n-1)$$

Bubble Sort

Given a list of N elements, repeat the following steps N-1 times:

- For each **pair** of adjacent numbers, if number on the left is greater than the number on the right, swap them.
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**.

16	16	22	22	22	22
6	12	18	18	17	22
6	12	12	14	17	22
6	8	12	14	17	22

Bubble Sort Analysis

BubbleSort (A)	cost	times
1. <code>n = Length[A];</code>	c_1	1
2. <code>for j = 0 to n-2</code>	c_2	$n-1$
3. <code>for i = 0 to n-j-2</code>	c_3	$\sum_{j=0}^{n-2} t_j$
4. <code>if A[i] > A[i+1]</code>	c_4	$\sum_{j=0}^{n-2} t_j$
5. <code>temp = A[i]</code>	c_5	$\sum_{j=0}^{n-2} t_j$
6. <code>A[i] = A[i+1]</code>	c_6	$\sum_{j=0}^{n-2} t_j$
7. <code>A[i+1] = temp</code>	c_7	$\sum_{j=0}^{n-2} t_j$
8. <code>return A;</code>	c_8	1

t_j : # of times the inner for loop statement is executed at iteration j

$$T(n) = c_1 + c_2(n-1) + c_3 \sum_{j=0}^{n-2} t_j + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7 \sum_{j=0}^{n-2} t_j + c_8$$

Best/Worst/Average Case Analysis

- Since we don't know at which iteration the list is completely sorted, the nested loop will execute completely.

$$\sum_{j=0}^{n-2} j = \frac{n(n-1)}{2}$$

$$T(n) = c_1 + c_2(n-1) + c_3 \frac{n(n-1)}{2} + c_4 \frac{n(n-1)}{2} + c_5 \frac{n(n-1)}{2} + c_6 \frac{n(n-1)}{2} + c_7 \frac{n(n-1)}{2} + c_8$$

$$= an^2 + bn + c$$

a quadratic function of n

- $T(n) = \Theta(n^2)$

order of growth in n^2

$$T(n) = c_1 + c_2(n-1) + c_3 \sum_{j=0}^{n-2} t_j + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7 \sum_{j=0}^{n-2} t_j + c_8$$

Bubble Sort Modified

BubbleSort(A)

```
1. n = Length[A];
2. for j = 0 to n-2
3.     swap = 0;
4.     for i = 0 to n-j-2
5.         if A[i] > A[i+1]
6.             temp = A[i]
7.             A[i] = A[i+1]
8.             A[i+1] = temp
           swap = swap + 1;
9.     if swap = 0
10.        break;
11. return A;
```

Best Case Analysis (Modified)

- The array is already sorted
 - Keep track of no. of swaps done in an iteration
 - Can finish early if **no swapping** occurs
 - $t_j = 1 \rightarrow n - 1$ comparisons in a single iteration

$$\begin{aligned}T(n) &= c_1 + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_8 \\&= (c_2 + c_3 + c_4)n + (c_1 - c_2 - c_3 - c_4 + c_8) \\&= an + b = \Theta(n)\end{aligned}$$

$$T(n) = c_1 + c_2(n-1) + c_3 \sum_{j=0}^{n-2} t_j + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7 \sum_{j=0}^{n-2} t_j + c_8$$

Worst Case Analysis (Modified)

- The array is in reverse sorted order

- Always $A[i] > A[i+1]$

$$\sum_{j=0}^{n-2} j = \frac{n(n-1)}{2}$$

$$T(n) = c_1 + c_2(n-1) + c_3 \frac{n(n-1)}{2} + c_4 \frac{n(n-1)}{2} + c_5 \frac{n(n-1)}{2} + c_6 \frac{n(n-1)}{2} + c_7 \frac{n(n-1)}{2} + c_8$$

$$= an^2 + bn + c$$

a quadratic function of n

$$T(n) = \Theta(n^2)$$

order of growth in n^2

Advantages

Simple to code

Requires little memory (in-place)

Disadvantages

$\Theta(n^2)$ running time in **worst** & **average** case

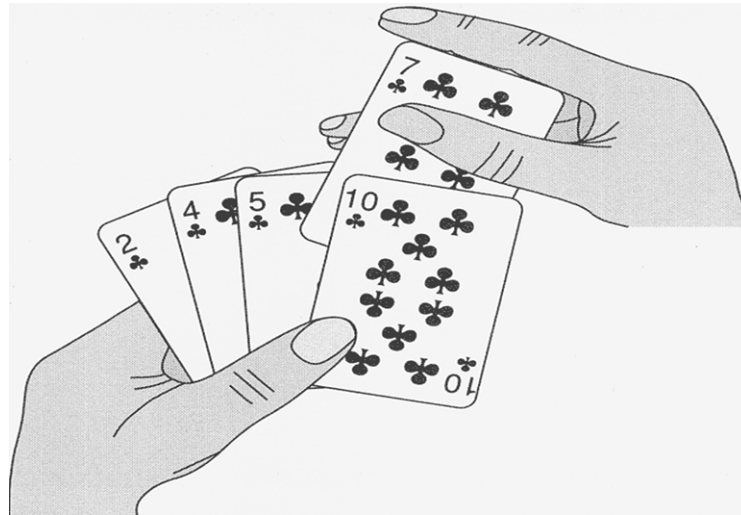
Nobody “EVER” uses bubble sort

(EXTREMELY INEFFICIENT)

$$T(n) = c_1 + c_2(n-1) + c_3 \sum_{j=0}^{n-2} t_j + c_4 \sum_{j=0}^{n-2} t_j + c_5 \sum_{j=0}^{n-2} t_j + c_6 \sum_{j=0}^{n-2} t_j + c_7 \sum_{j=0}^{n-2} t_j + c_8$$

Insertion Sort

- Idea: like sorting a hand of playing cards
 - Start with empty left hand and cards face down on the table.
 - Remove one card at a time from the table, and insert it into the correct position in the left hand
 - compare it with each card already in the hand, from right to left
 - The cards held in the left hand are sorted
 - these cards were originally the top cards of the pile on the table



Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	2.78	7.42	0.56	1.12	1.17	0.32	6.21	4.42	3.14	7.71

Iteration 0: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	2.78	7.42	0.56	1.12	1.17	0.32	6.21	4.42	3.14	7.71

Iteration 1: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	2.78	0.56	7.42	1.12	1.17	0.32	6.21	4.42	3.14	7.71



Iteration 2: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	2.78	7.42	1.12	1.17	0.32	6.21	4.42	3.14	7.71



Iteration 2: step 1.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	2.78	7.42	1.12	1.17	0.32	6.21	4.42	3.14	7.71

Iteration 2: step 2.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	2.78	1.12	7.42	1.17	0.32	6.21	4.42	3.14	7.71



Iteration 3: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	2.78	7.42	1.17	0.32	6.21	4.42	3.14	7.71



Iteration 3: step 1.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	2.78	7.42	1.17	0.32	6.21	4.42	3.14	7.71

Iteration 3: step 2.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	2.78	1.17	7.42	0.32	6.21	4.42	3.14	7.71



Iteration 4: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	1.17	2.78	7.42	0.32	6.21	4.42	3.14	7.71



Iteration 4: step 1.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	1.17	2.78	7.42	0.32	6.21	4.42	3.14	7.71

Iteration 4: step 2.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	1.17	2.78	0.32	7.42	6.21	4.42	3.14	7.71



Iteration 5: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	1.17	0.32	2.78	7.42	6.21	4.42	3.14	7.71



Iteration 5: step 1.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	1.12	0.32	1.17	2.78	7.42	6.21	4.42	3.14	7.71



Iteration 5: step 2.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.56	0.32	1.12	1.17	2.78	7.42	6.21	4.42	3.14	7.71



Iteration 5: step 3.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	7.42	6.21	4.42	3.14	7.71



Iteration 5: step 4.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	7.42	6.21	4.42	3.14	7.71

Iteration 5: step 5.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	6.21	7.42	4.42	3.14	7.71



Iteration 6: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	6.21	7.42	4.42	3.14	7.71

Iteration 6: step 1.

Insertion Sort

`InsertionSort(A, n)`

1. `for i = 1 to n-1`

2. `key = A[i]`

3. `j = i - 1`

4. `while (j >= 0) and (A[j] > key)`

5. `A[j+1] = A[j]`

6. `j = j - 1`

7. `A[j+1] = key`

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	6.21	4.42	7.42	3.14	7.71



Iteration 7: **step 0.**

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	4.42	6.21	7.42	3.14	7.71



Iteration 7: step 1.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	4.42	6.21	7.42	3.14	7.71

Iteration 7: step 2.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	4.42	6.21	3.14	7.42	7.71



Iteration 8: step 0.

Insertion Sort

`InsertionSort(A, n)`

1. `for i = 1 to n-1`

2. `key = A[i]`

3. `j = i - 1`

4. `while (j >= 0) and (A[j] > key)`

5. `A[j+1] = A[j]`

6. `j = j - 1`

7. `A[j+1] = key`

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	4.42	3.14	6.21	7.42	7.71



Iteration 8: **step 1.**

Insertion Sort

`InsertionSort(A, n)`

1. `for i = 1 to n-1`

2. `key = A[i]`

3. `j = i - 1`

4. `while (j >= 0) and (A[j] > key)`

5. `A[j+1] = A[j]`

6. `j = j - 1`

7. `A[j+1] = key`

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	3.14	4.42	6.21	7.42	7.71



Iteration 8: **step 2.**

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	3.14	4.42	6.21	7.42	7.71

Iteration 8: step 3.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	3.14	4.42	6.21	7.42	7.71

Iteration 9: step 0.

Insertion Sort

InsertionSort(A, n)

1. for i = 1 to n-1

2. key = A[i]

3. j = i - 1

4. while (j >= 0) and (A[j] > key)

5. A[j+1] = A[j]

6. j = j - 1

7. A[j+1] = key

Array index	0	1	2	3	4	5	6	7	8	9
Value	0.32	0.56	1.12	1.17	2.78	3.14	4.42	6.21	7.42	7.71

Iteration 10: **DONE!**

Insertion Sort Analysis

	cost	times
<code>InsertionSort(A, n) {</code>		
<code>for i = 1 to n-1 {</code>	c_1	$n-1$
<code>key = A[i]</code>	c_2	$n-1$
<code>j = i - 1;</code>	c_3	$n-1$
<code>while (j >= 0) and (A[j] > key) {</code>	c_4	$\sum_{i=1}^{n-1} t_i$
<code>A[j+1] = A[j]</code>	c_5	$\sum_{i=1}^{n-1} t_i$
<code>j = j - 1</code>	c_6	$\sum_{i=1}^{n-1} t_i$
<code>}</code>		
<code>A[j+1] = key</code>	c_7	$n-1$
<code>}</code>		
<code>}</code>		
t_i : # of times the while statement is executed at iteration i		

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} t_i + c_6 \sum_{i=1}^{n-1} t_i + c_7(n-1)$$

Best Case Analysis

“while j >= 0 and A[j] > key”

- The array is already sorted in ascending order
- Inner loop will not be executed
- The number of moves: $2*(n-1)$
- The number of key comparisons: $(n-1)$

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1)$$

$$= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_1 + c_2 + c_3 + c_4 + c_7)$$

$$= an + b = \Theta(n)$$

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} t_i + c_6 \sum_{i=1}^{n-1} t_i + c_7(n-1)$$

Worst Case Analysis

“while $j \geq 0$ and $A[j] > \text{key}$ ”

- The array is in reverse sorted order
- The number of moves: $2 \cdot (n-1) + (1+2+\dots+n-1) = 2 \cdot (n-1) + n \cdot (n-1)/2$
- The number of key comparisons: $(1+2+\dots+n-1) = n \cdot (n-1)/2$
 - Always $A[j] > \text{key}$ in **while** loop test
 - Have to compare **key** with all elements to the left of the j -th position
 $\Rightarrow$ compare with $i-1$ elements $\Rightarrow t_j = i$

$$\sum_{i=1}^{n-1} t_i = \frac{n(n-1)}{2}$$

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + (c_4 + c_5 + c_6) \left(\frac{n(n-1)}{2} \right) + c_7(n-1)$$

$$= an^2 + bn + c$$

a quadratic function of n
order of growth in n^2

$$T(n) = \Theta(n^2)$$

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} t_i + c_6 \sum_{i=1}^{n-1} t_i + c_7(n-1)$$

Insertion Sort Summary

- Running time not only depends on the size of the array but also the contents of the array.
- Average-case: $\rightarrow O(n^2)$
 - We have to look at all possible initial data organizations.
- Advantages
 - Good running time for “almost sorted” arrays $\Theta(n)$
- Disadvantages
 - $\Theta(n^2)$ running time in **worst** and **average** case
 - $\approx n^2/2$ **comparisons** and **exchanges**

Types of Sorting Algorithms

- Non-recursive/incremental comparison sorting
 - Selection sort
 - Bubble sort
 - Insertion sort
- Recursive comparison sorting
 - Merge sort
 - Quick sort
 - Heap sort
- Non-comparison linear sorting
 - Count sort
 - Radix sort
 - Bucket sort

Recursive Comparison Sorting

There are many ways to design algorithms:

Insertion/Bubble/Selection sort uses an incremental approach

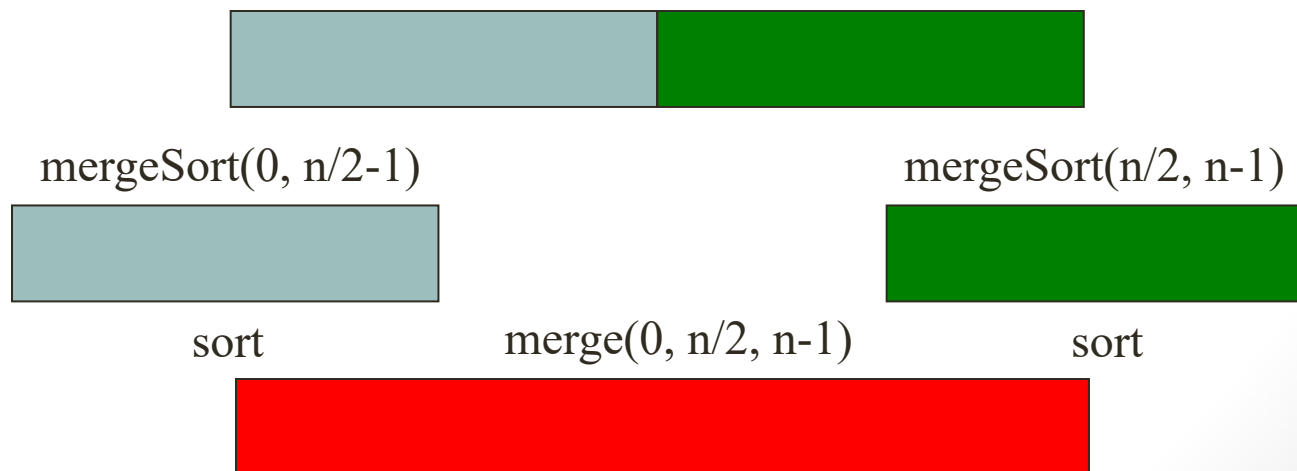
- Having sorted to array $A[i..j]$
- We insert the single element $A[j+1]$ into its proper place, to get a sorted array $A[i..j+1]$

An alternative design approach is “Divide and Conquer”

- **Divide** the problems into a number of sub-problems
- **Conquer** the sub-problems by solving them recursively. If sub-problem sizes are small enough, just solve them in a straight forward manner.
- **Combine** the solutions to the sub-problems into the solution for the original problem.

Merge Sort

- **Idea:** Order a list of values by recursively dividing the list in half until each sub-list has one element, then recombining
- More specifically:
 - **Divide** the n element sequence to be sorted into two subsequences of $n/2$ elements each.
 - **Conquer:** Sort the two subsequences to produce the sorted answer.
 - **Combine:** Merge the two sorted sub sequences to produce the sorted answer.



Merge Sort Pseudocode

MERGE-SORT(A, p, r)

1 **if** $p < r$

2 $q = \lfloor (p + r) / 2 \rfloor$

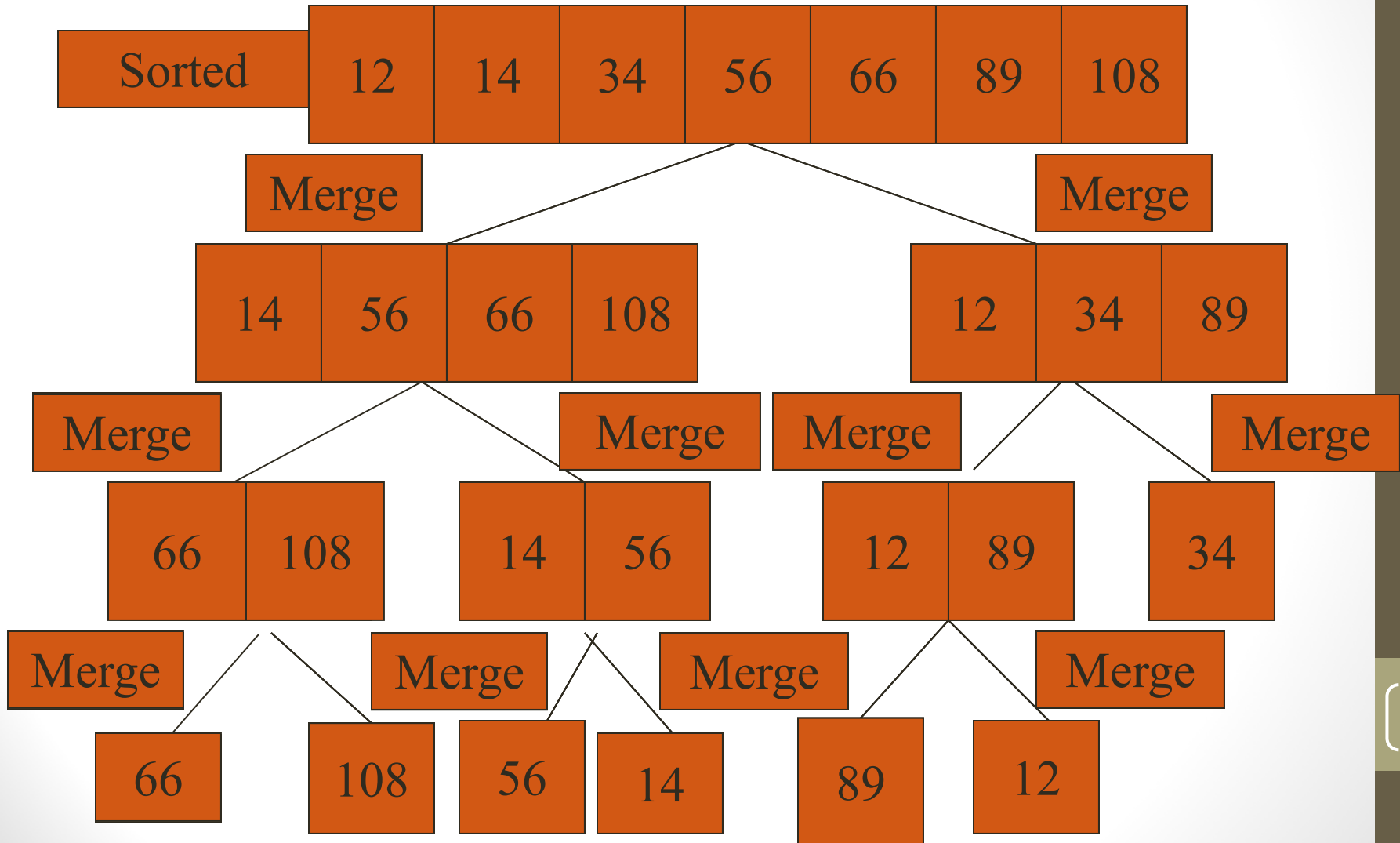
3 MERGE-SORT(A, p, q)

4 MERGE-SORT($A, q + 1, r$)

5 MERGE(A, p, q, r)

Merge Sort

Base Case: When the sequences to be sorted has length 1.



Merge Function

Given two sorted arrays, *merge* operation produces a sorted array with all the elements of the two arrays



Divide the array in half and conquer



Merge the halves

tempArray:



copy

Original Array:



Merge Function Pseudocode

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Complexity of *Merge Function*?

$O(n)$

Merge Sort Analysis

MERGE-SORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \lfloor (p + r)/2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
```

Let $T(n)$ be the time taken by this algorithm to sort an array of n elements dividing A into sub-arrays A_1 and A_2 . Merge operation takes the linear time. Consequently,

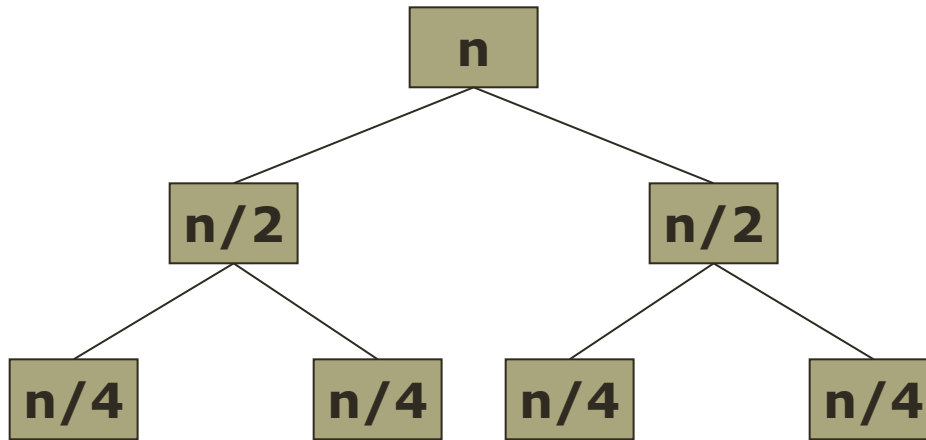
$$T(n) = T(n/2) + T(n/2) + \theta(n)$$

$$T(n) = 2T(n/2) + \theta(n)$$

The above recurrence relation is non-homogenous and can be solved by any of the methods:

- Substitution
- Recursion tree
- Master method

Merge Sort Analysis



Level 0 Merge n items:
 $O(n)$

Level 1 Merge two $n/2$ items:
 $O(n)$

Level 2

Each level requires $O(n)$ operations



Tree Height : $\log_2 n$

Each level $O(n)$ operations & $O(\log_2 n)$ levels $\rightarrow O(n * \log_2 n)$

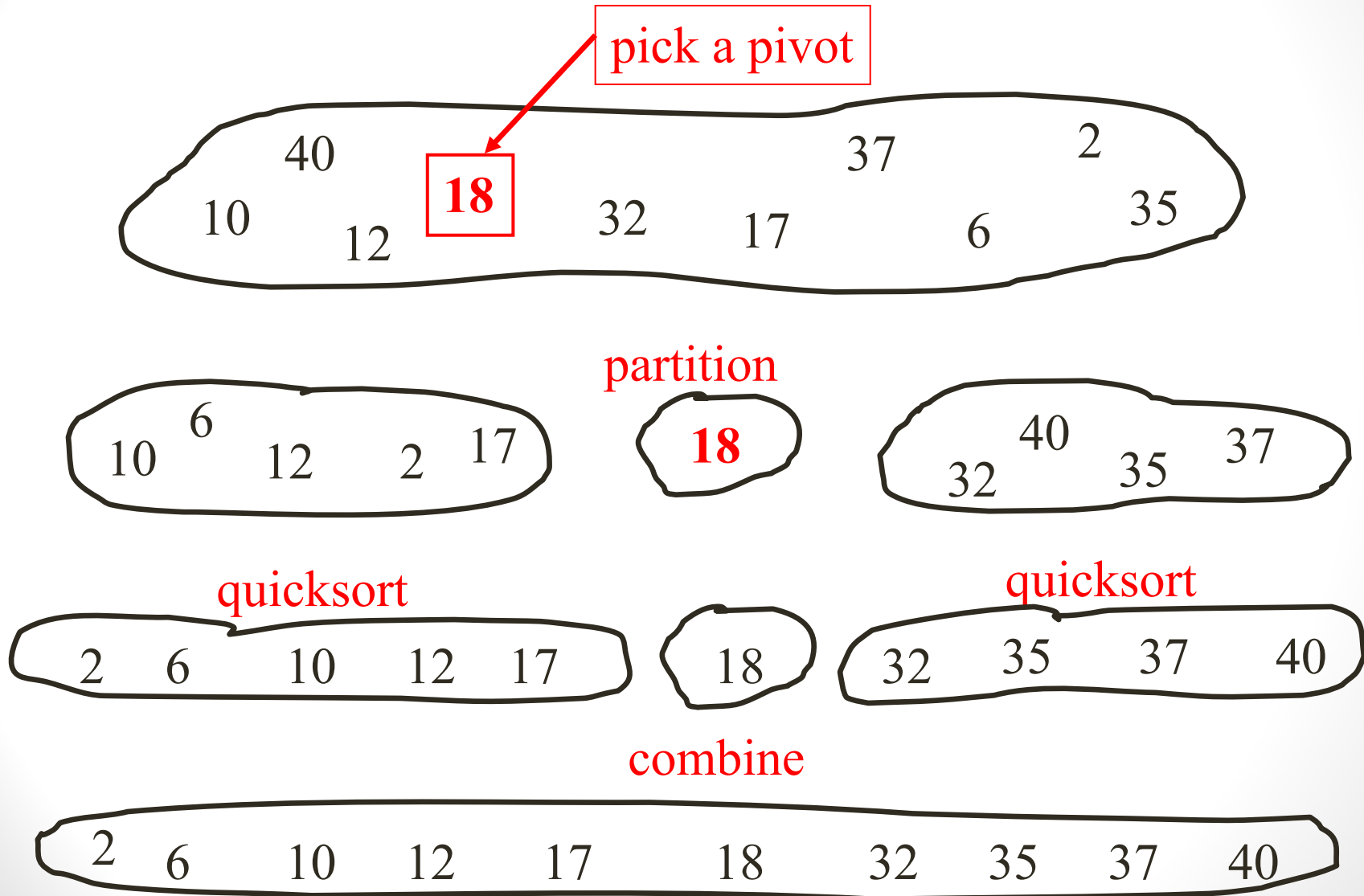
Merge Sort Analysis

- Worst case: $O(n * \log_2 n)$.
- Average case: $O(n * \log_2 n)$.
- Performance is independent of the initial order of the array items.
- Advantage:
 - Mergesort is an extremely fast algorithm.
- Disadvantage:
 - Mergesort requires a second array as large as the original array.

Quick Sort

- **Quick Sort:** orders a list of values by partitioning the list around one element called a **pivot**, then sorting each partition. It is based on divide and conquer approach.
- Key Points: Given an array **S** to be sorted
 - Pick any element **v** in array **S** as the **pivot** (i.e. **partition element**)
 - Partition the remaining elements in **S** into two groups
 - **S1** = {all elements in **S** – {**v**} that are smaller than **v**}
 - **S2** = {all elements in **S** – {**v**} that are larger than **v**}
 - apply the quick sort algorithm (**recursively**) to **both partitions**
- Trick lies in handling the partitioning
 - i.e. picking a good pivot is essential to performance

Quick Sort Illustrated



Quick Sort Pseudocode

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q = \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)

Partitioning Algorithm

Original input : $S = \{6, 1, 4, 9, 0, 3, 5, 2, 7, 8\}$

Pick the first element as pivot

8 1 4 9 0 3 5 2 7 **6**
pivot

Have two 'iterators' – **i** and **j**

- **i** starts at first element and moves forward
- **j** starts at last element and moves backwards

8 1 4 9 0 3 5 2 **7** 6
i j pivot

While ($i < j$)

- Move **i** to the right till we find a number greater than **pivot**
- Move **j** to the left till we find a number smaller than **pivot**

If ($i < j$) **swap**($S[i]$, $S[j]$)

- The effect is to push larger elements to the right and smaller elements to the left

Swap the **pivot** with $S[i]$

Partitioning Algorithm Illustrated

	i							j	pivot	
	8	1	4	9	0	3	5	2	7	6
Move	i							j	pivot	
	8	1	4	9	0	3	5	2	7	6
swap	i							j	pivot	
	2	1	4	9	0	3	5	8	7	6
move				i			j		pivot	
	2	1	4	9	0	3	5	8	7	6
swap				i			j		pivot	
	2	1	4	5	0	3	9	8	7	6
move						j	i		pivot	i and j have crossed
	2	1	4	5	0	3	9	8	7	6
Swap S[i] with pivot	2	1	4	5	0	3	6	8	7	9
						j	i pivot			

Partitioning Pseudocode

PARTITION(A, p, r)

*What is the running time of
partition()?*

1. pivot = A[p];
2. leftPointer = p + 1
3. rightPointer = r
4. while (True)
5. while (A[leftPointer] < pivot)
6. leftPointer++;
7. while (A[rightPointer] >= pivot)
8. rightPointer--;
9. if leftPointer >= rightPointer
10. break;
11. else
12. A[leftPointer] $\leftrightarrow$ A[rightPointer]
13. A[leftPointer] $\leftrightarrow$ pivot

***partition()** runs in $O(n)$ time*

Partitioning Algorithm Illustrated

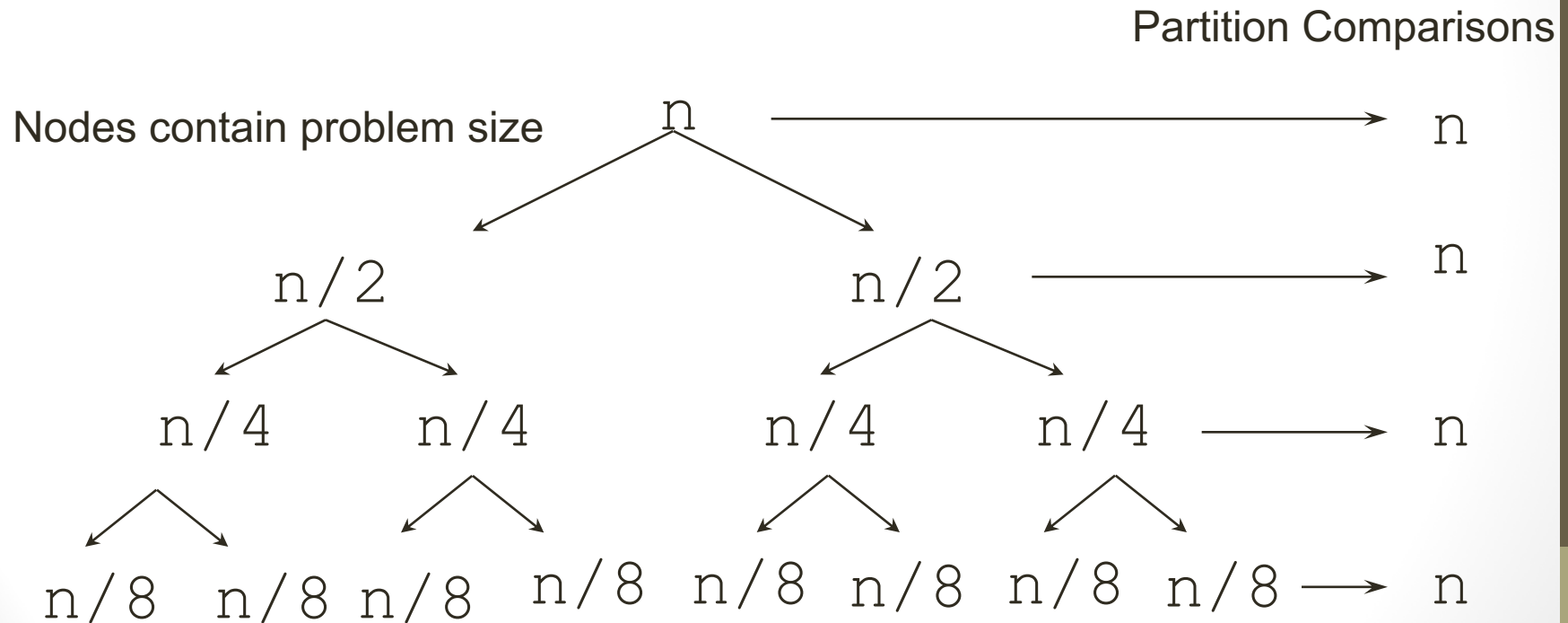
- choose pivot: 4 3 6 9 2 4 3 1 2 1 8 9 3 5 6
- search: 4 3 6 9 2 4 3 1 2 1 8 9 3 5 6
- swap: 4 3 3 9 2 4 3 1 2 1 8 9 6 5 6
- search: 4 3 3 9 2 4 3 1 2 1 8 9 6 5 6
- swap: 4 3 3 1 2 4 3 1 2 9 8 9 6 5 6
- search: 4 3 3 1 2 4 3 1 2 9 8 9 6 5 6
- swap: 4 3 3 1 2 2 3 1 4 9 8 9 6 5 6
- search: 4 3 3 1 2 2 3 1 4 9 8 9 6 5 6 (left > right)
- swap with pivot: 1 3 3 1 2 2 3 4 4 9 8 9 6 5 6

Quick Sort: Best Case Analysis

Assuming that keys are random, uniformly distributed.

The best case running time occurs when pivot is the median

- Partition splits array in two sub-arrays of size $n/2$ at each step



$$T(n) = 2 T(n/2) + cn$$

$$T(n) = cn \log n + n = O(n \log n)$$

Quick Sort: Worst Case Analysis

Assuming that keys are random, uniformly distributed.

The worst case running time occurs when pivot is the smallest (or largest) element all the time

- One of the two sub-arrays will have zero elements at each step

Height of Tree = n

$$\text{Total Cost} = \sum_{i=1}^{n-1} t_i = \frac{n(n-1)}{2}$$

$$T(n) = T(n-1) + cn$$

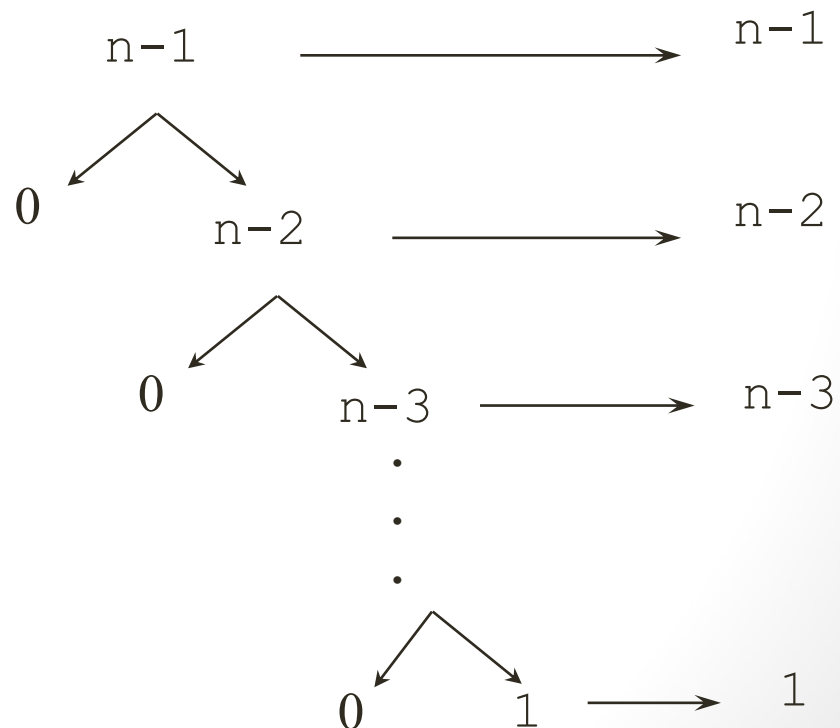
$$T(n-1) = T(n-2) + c(n-1)$$

$$T(n-2) = T(n-3) + c(n-2)$$

...

$$T(2) = T(1) + 2c$$

$$T(N) = T(1) + c \sum_{i=2}^N i = O(N^2)$$



Quick Sort: Average Case Analysis

Assuming that keys are random, uniformly distributed.

The average case running time occurs when pivot can take any position at each step

- The size of two sub-arrays will vary at each step

Per level Cost = n

Height of Tree = $\log_x n$ (x depends on how the partition is split)

$$O(n \lg n)$$

Picking the Pivot

- How would you pick one?
- Strategy 1: Pick the **first element** in **S**
 - Works only if input is random
 - What if input **S** is sorted, or even mostly sorted?
 - All the remaining elements would go into either **S1** or **S2**!
 - Terrible performance!
 - Why worry about sorted input?
 - Remember → Quicksort is recursive, so sub-problems could be sorted
 - Plus mostly sorted input is quite frequent

Picking the Pivot (contd.)

- Strategy 2: Pick the pivot **randomly**
 - Would usually work well, even for mostly sorted input
 - Unless the random number generator is not quite random!
 - Plus random number generation is an expensive operation
- Strategy 3: **Median-of-three Partitioning**
 - *Ideally*, the pivot should be the **median** of input array **S**
 - Median = element in the middle of the sorted sequence
 - Would divide the input into two almost equal partitions
 - Unfortunately, harder to calculate median quickly, without sorting first!
 - So find the approximate median
 - Pivot = median of the **left-most**, **right-most** and **center** elements of array **S**
 - Solves the problem of sorted input

Picking the Pivot (contd.)

- Example: **Median-of-three Partitioning**
 - Let input $S = \{6, 1, 4, 9, 0, 3, 5, 2, 7, 8\}$
 - $\text{left} = 0$ and $S[\text{left}] = 6$
 - $\text{right} = 9$ and $S[\text{right}] = 8$
 - $\text{center} = (\text{left} + \text{right})/2 = 4$ and $S[\text{center}] = 0$
 - Pivot
 - = Median of $S[\text{left}]$, $S[\text{right}]$, and $S[\text{center}]$
 - = median of 6, 8, and 0
 - = $S[\text{left}] = 6$

Quick Sort: Final Comments

- What happens when the array contains many duplicate elements?
 - Not stable
- What happens when the size of the array is small?
 - For small arrays (**$N \leq 50$**), Insertion sort is faster than quick sort due to recursive function call overheads of quick sort. Use hybrid algorithm; quick sort followed by insertion sort when **$N \leq 50$**
- However, Quicksort is *usually* $O(n \log_2 n)$
 - Constants are so good that it is generally the fastest algorithm known
 - Most real-world sorting is done by Quicksort
 - For optimum efficiency, the pivot must be chosen carefully
 - “Median of three” is a good technique for choosing the pivot
- However, no matter what you do, some bad cases can be constructed where Quick Sort runs in $O(n^2)$ time

Heap Sort

Uses *Heap* data structure to manage data during algorithm execution

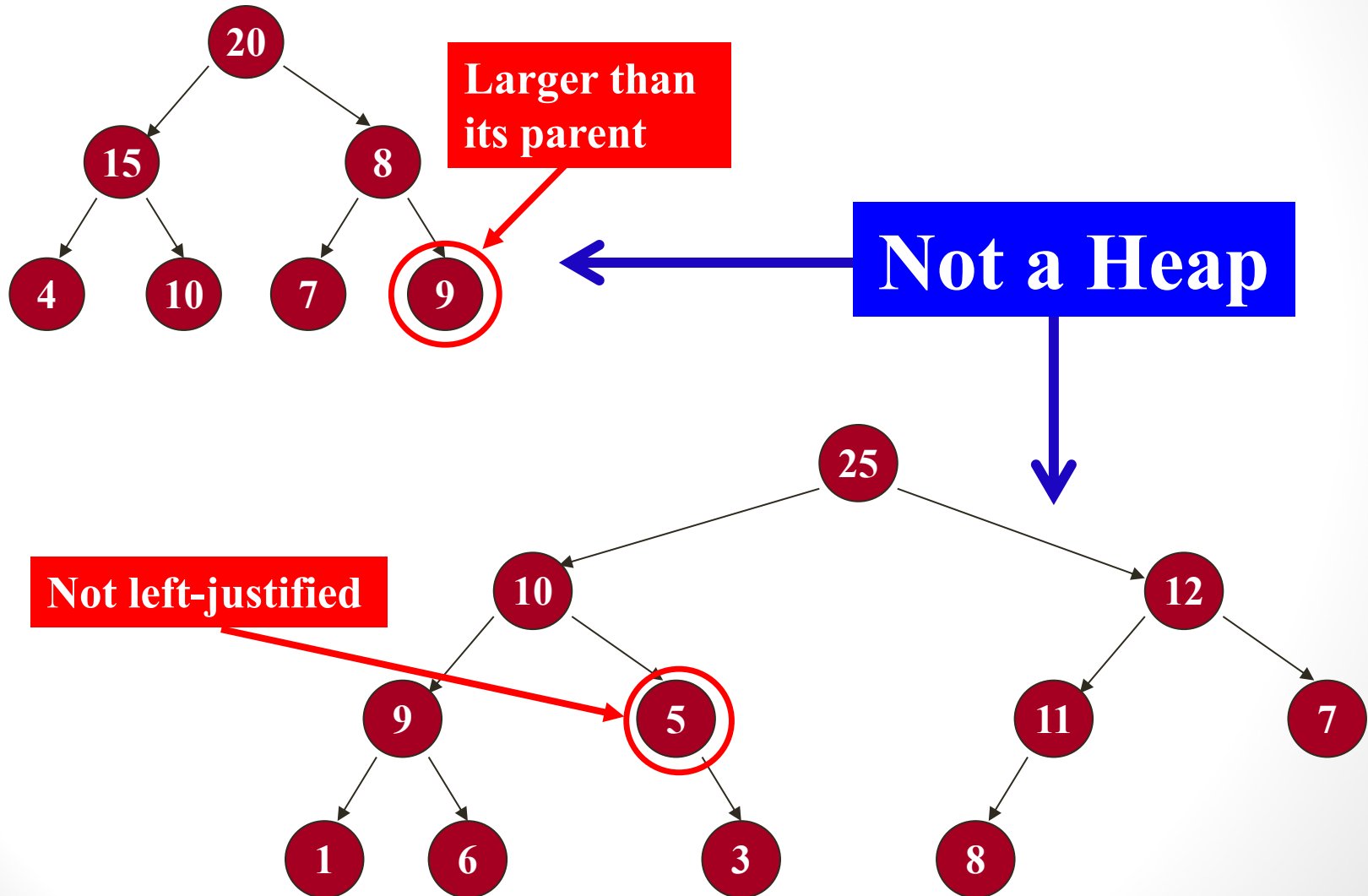
A *(Max/Min)-Heap* is a balanced, left-justified binary tree in which no node has a value *greater/lesser* than the value in its parent

- $A[\text{parent}[i]] \geq A[i]$ (max-heap)
- $A[\text{parent}[i]] \leq A[i]$ (min-heap)

Largest/Smallest element is *stored at the root*.

Subtree rooted at a node contains values no *larger/smaller* than that at the node.

Heap Property

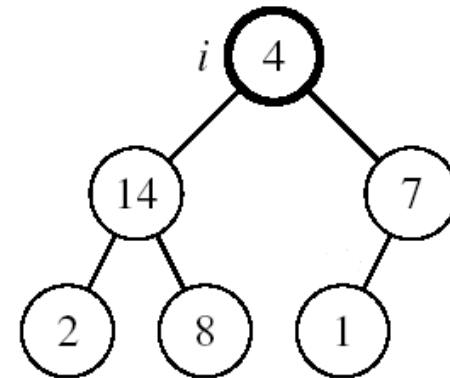


Maintaining the Heap Property

Suppose a node is smaller than a child, violating heap property. To maintain heap property, exchange the node with the larger child, moving the node down the tree until it is not smaller than its children

Alg: MAX-HEAPIFY(A, i, n)

1. $l \leftarrow \text{LEFT}(i)$
2. $r \leftarrow \text{RIGHT}(i)$
3. **if** $l \leq n$ and $A[l] > A[i]$
4. **then** $\text{largest} \leftarrow l$
5. **else** $\text{largest} \leftarrow i$
6. **if** $r \leq n$ and $A[r] > A[\text{largest}]$
7. **then** $\text{largest} \leftarrow r$
8. **if** $\text{largest} \neq i$
9. **then** exchange $A[i]$ $A[\text{largest}]$
10. MAX-HEAPIFY($A, \text{largest}, n$)



*Complexity of **MAX-HEAPIFY** () ?*

$O(\lg n)$ where $\lg n$ is the height of the tree

Array Representation of Heaps

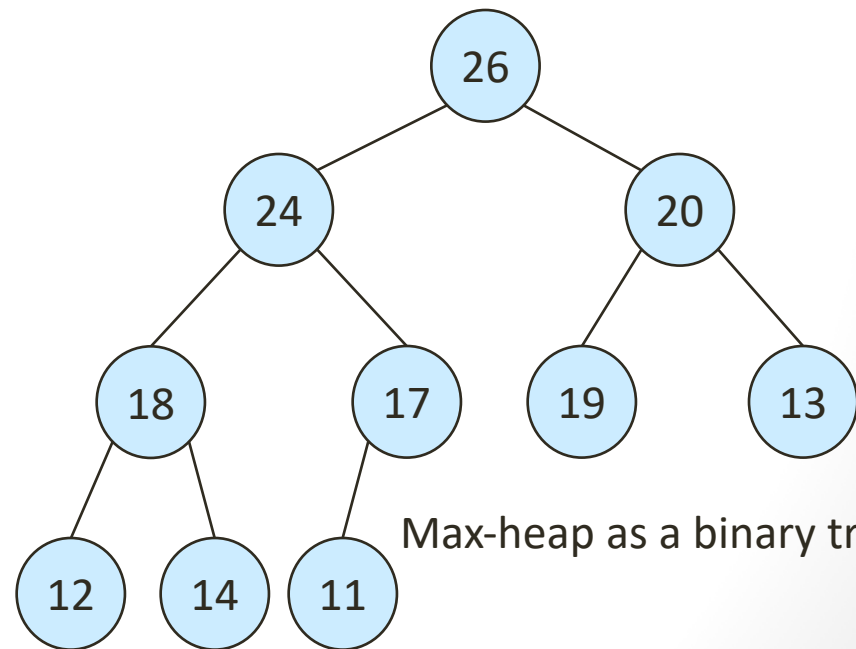
A heap can be stored as an array A .

- Root of tree is $A[1]$
- Left child of $A[i] = A[2i]$
- Right child of $A[i] = A[2i + 1]$
- Parent of $A[i] = A[\lfloor i/2 \rfloor]$
- $\text{Heapsize}[A] \leq \text{length}[A]$

The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) .. n]$ are leaves

1	2	3	4	5	6	7	8	9	10
26	24	20	18	17	19	13	12	14	11

Max-heap as an array



Max-heap as a binary tree

Node Insertion: Insert in last row from left to right

Node Deletion: Delete from last row from right to left

Building a Heap

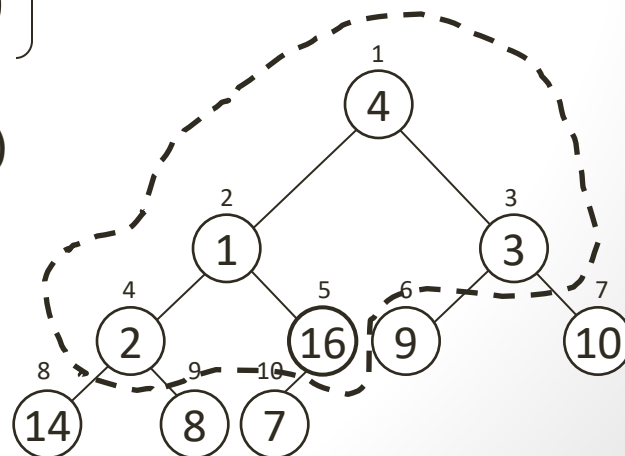
A given array containing n elements can be converted into a max-heap by applying MAX-HEAPIFY on all interior nodes

- The elements in the subarray $A[1 \dots \lfloor n/2 \rfloor]$ are interior nodes
- The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ are leaves

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A]$
 2. **for** $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1
 3. **do** MAX-HEAPIFY(A, i, n) $O(\lg n)$
- $\left. \begin{array}{l} \text{ } \\ \text{ } \\ \text{ } \end{array} \right\} O(n)$

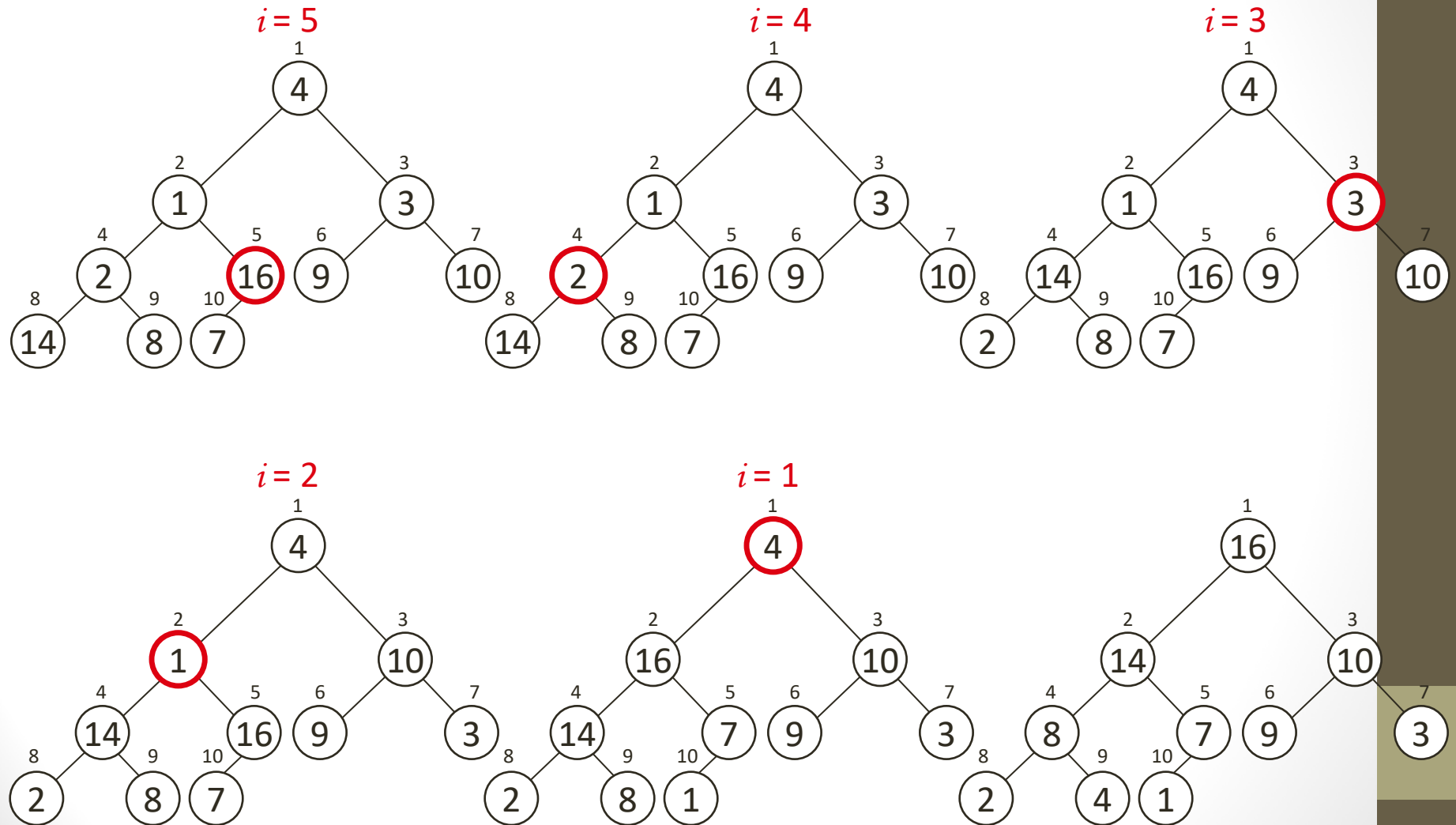
Complexity of BUILD-MAX-HEAP? $O(n \lg n)$



Example:

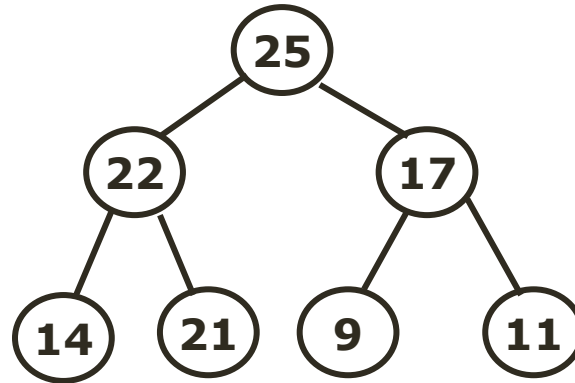
A:

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



Heap Sort

A given array which has been heapified does not mean it is sorted

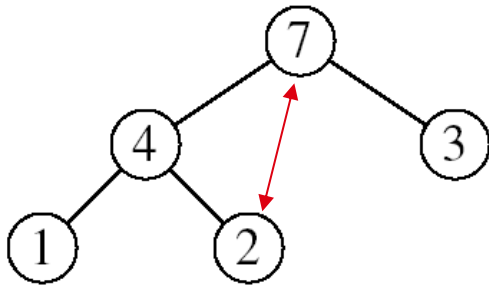


Notice that the largest number is at the root. It can be swapped with the rightmost leaf at the deepest level (its original position) and heap size decreased.

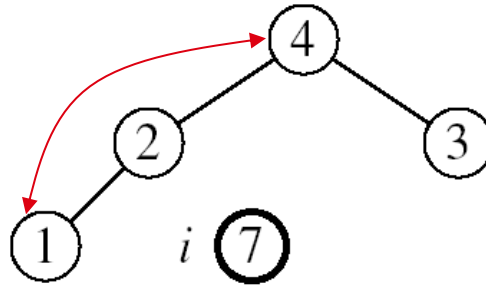
The new root has lost heap property. **Heapify** the new root and repeat this process until only one node remain.

Example:

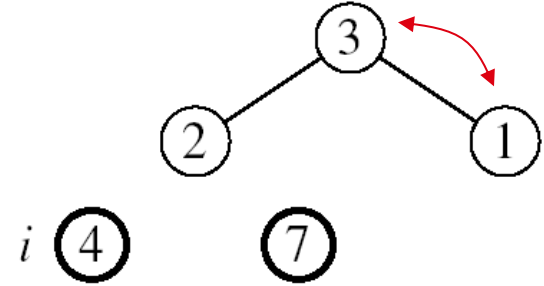
$A=[7, 4, 3, 1, 2]$



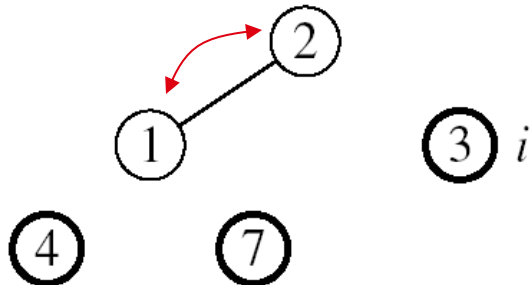
MAX-HEAPIFY(A, 1, 4)



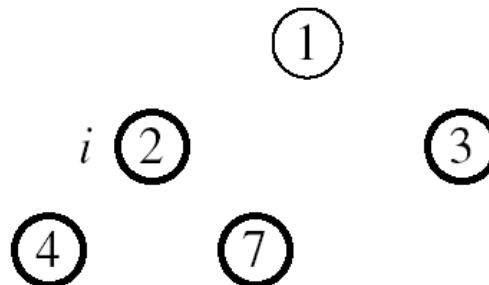
MAX-HEAPIFY(A, 1, 3)



MAX-HEAPIFY(A, 1, 2)



MAX-HEAPIFY(A, 1, 1)



A

1	2	3	4	7
---	---	---	---	---

Heap Sort Pseudocode

HEAPSORT(A)

1. BUILD-MAX-HEAP(A) $O(n \lg n)$
 2. for $i \leftarrow \text{length}[A]$ downto 2
 3. do exchange $A[1]$ $A[i]$ $O(\lg n)$
 4. MAX-HEAPIFY(A, 1, $i - 1$)
- } $n-1$ times

*What is the running time of **HEAPSORT** ()? $O(n \lg n)$*

Why Heap Sort?

- Heap Sort is always $O(n \lg n)$ and is in-place
- Quicksort is usually $O(n \lg n)$ but in the worst case slows to $O(n^2)$
- Quicksort is generally faster, but Heapsort is better in time-critical applications

How Fast Can We Sort?

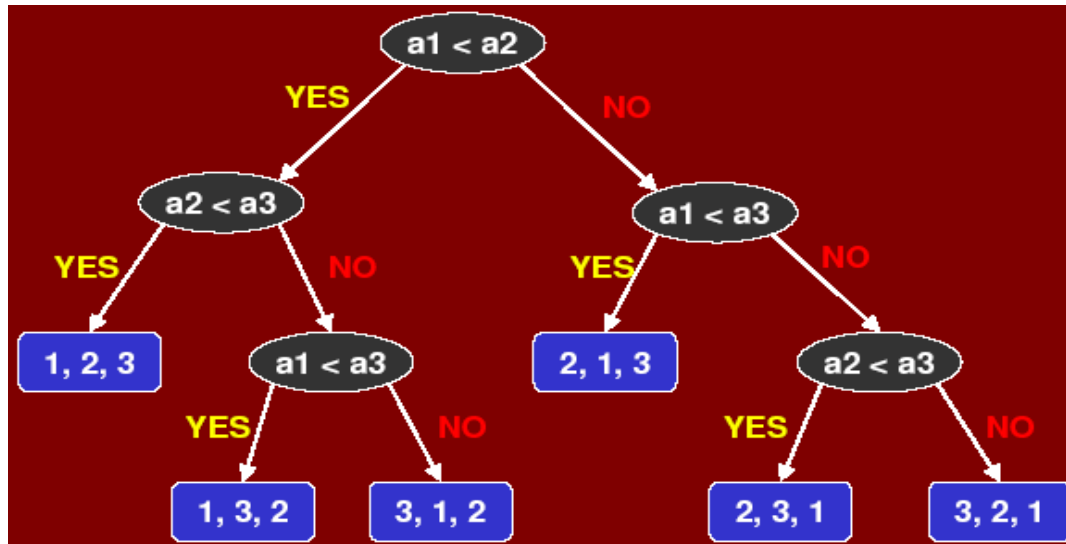
- Selection Sort, Bubble Sort, Insertion Sort $\rightarrow O(n^2)$
- Merge Sort, Quick Sort, Heap Sort $\rightarrow O(n \lg n)$
- What is common to all these algorithms?
 - Make **comparisons** between input elements
- Lower bound for comparison based sorting
 - **Theorem:** To sort n elements, comparison sorts **must** make $\Omega(n \lg n)$ comparisons in the worst case.
 - **Proof:** Draw decision tree and calculate asymptotic height of any decision tree for sorting n elements

Lower Bound For Comparison Sorting

Consider sorting three numbers a_1, a_2, a_3 .

There are $3! = 6$ possible combinations:

$(a_1, a_2, a_3), (a_1, a_3, a_2), (a_3, a_2, a_1), (a_3, a_1, a_2), (a_2, a_1, a_3), (a_2, a_3, a_1)$



- What's the minimum # of leaves? $n!$
- What's the maximum # of leaves of a binary tree of height h ? 2^h
- minimum # of leaves $\leq$ maximum # of leaves

Lower Bound For Comparison Sorting

- So we have...

$$n! \leq 2^h$$

- Taking logarithms:

$$\lg(n!) \leq h$$

- Stirling's approximation tells us:

$$n! > \left(\frac{n}{e}\right)^n \quad h \geq \lg\left(\frac{n}{e}\right)^n$$

$$= n \lg n - n \lg e$$

$$= \Omega(n \lg n)$$

- Thus the minimum height of a decision tree is $\Omega(n \lg n)$ therefore the time to comparison sort n elements is $\Omega(n \lg n)$

- Corollary: Quick Sort, Heap Sort and Merge Sort are asymptotically optimal comparison sorts. *Can we do any better than $\Omega(n \lg n)$?*

Types of Sorting Algorithms

- Non-recursive/incremental comparison sorting
 - Selection sort
 - Bubble sort
 - Insertion sort
- Recursive comparison sorting
 - Merge sort
 - Quick sort
 - Heap sort
- Non-comparison linear sorting
 - Count sort
 - Radix sort
 - Bucket sort

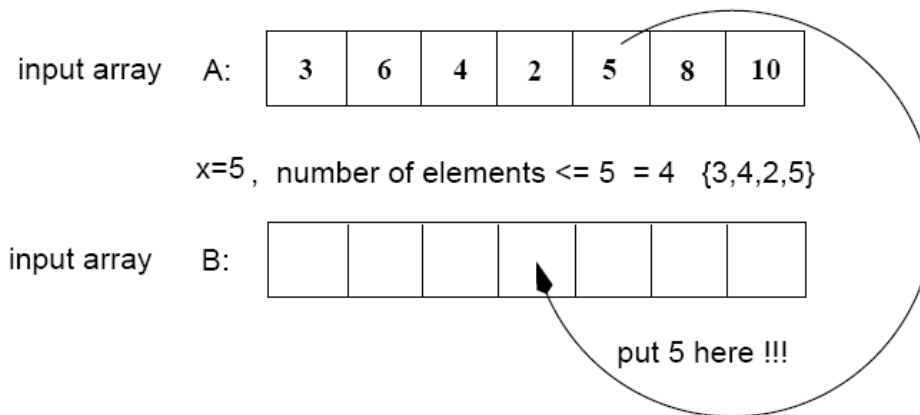
Counting Sort

- **Assumptions:**

- n integers which are in the range $[0, k]$
- k is in the order of n , that is, $k = O(n)$.

- **Idea:**

- For each element x , find the number of elements $\leq x$
- Place x into its correct position in the output array



Counting Sort

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

Input Array

	0	1	2	3	4	5
C	2	0	2	3	0	1

Auxiliary Array

	0	1	2	3	4	5
C	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
B	0	0	2	2	3	3	3	5

Output Array

	0	1	2	3	4	5
C	0	2	2	4	7	7

Counting Sort

Counting-Sort(A, B, k)

 for $i = 1$ to k

 do $C[i] = 0$

$\Theta(k)$

 for $j = 1$ to $\text{length}[A]$

 do $C[A[j]] = C[A[j]] + 1$

$\Theta(n)$

// $C[i]$ now contains the number of elements $= i$

 for $i = 2$ to k

 do $C[i] = C[i] + C[i-1]$

$\Theta(k)$

// $C[i]$ now contains the number of elements $\leq i$

 for $j = \text{length}[A]$ downto 1

 do $B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

$\Theta(n)$

Running time is $\Theta(n+k)$ (or $\Theta(n)$ if $k = O(n)$)!

Counting Sort

- Is counting sort stable
 - The input order is maintained among items with equal keys
- Cool! *Why don't we always use counting sort?*
 - Because it depends on range k of elements
- *Could we use counting sort to sort 32 bit integers? Why or why not?*
 - Answer: no, k too large ($2^{32} = 4,294,967,296$)

Radix Sort

- Represents keys as d -digit numbers in some base- k
 - $\text{key} = x_1x_2\dots x_d$ where $0 \leq x_i \leq k-1$
- Example: $\text{key}=15$
 - $\text{key}_{10} = 15$, $d=2$, $k=10$ where $0 \leq x_i \leq 9$
 - $\text{key}_2 = 1111$, $d=4$, $k=2$ where $0 \leq x_i \leq 1$
- Assumptions: $d=O(1)$ and $k=O(n)$
- Sorting looks at one column at a time
 - For a d digit number, sort the least significant digit first
 - Continue sorting on the next least significant digit, until all digits have been sorted
 - Requires only d passes through the list

326
453
608
835
751
435
704
690

Radix Sort

RadixSort(A, d)

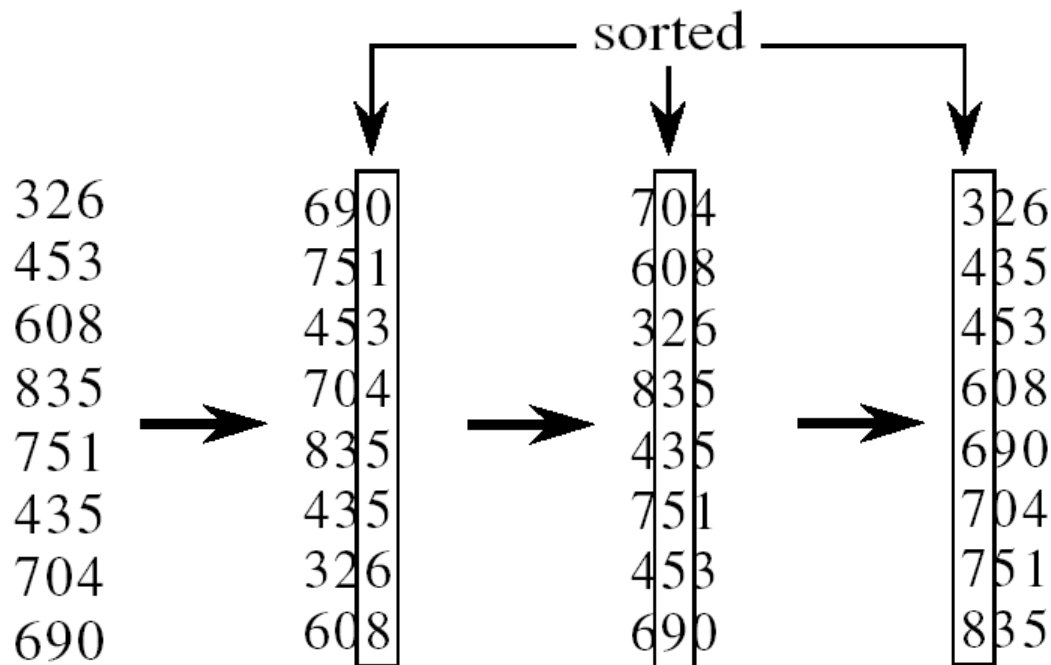
 for i=1 to d

StableSort(A) on digit i

$\Theta(n+k)$

d times

Running time is $\Theta(dn+dk)$ (or $\Theta(n)$ if $k = O(n)$ and $d = O(1)$)



(stable sort: preserves order of identical elements)

Radix Sort

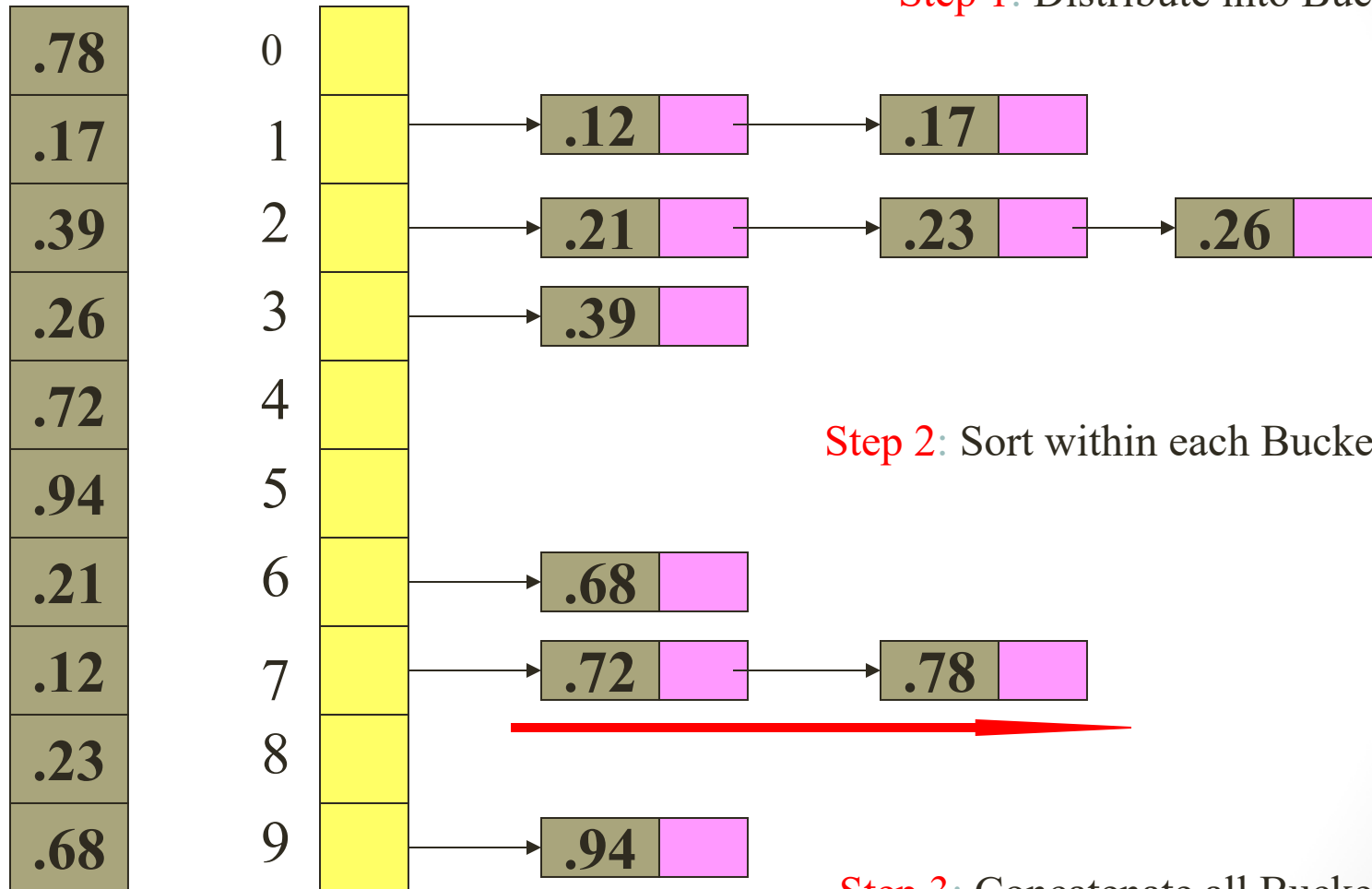
- Problem: sort 1 million 64-bit numbers
 - Treat as four-digit radix 2^{16} numbers
 - Can sort in just four passes with radix sort!
- *So why would we ever use anything but radix sort?*

Bucket Sort

- Assumption:
 - Input numbers are **uniformly distributed** in $[0,1)$.
 - Suppose input size is n .
- Idea:
 - Divide $[0,1)$ into n equal-sized buckets ($k = \Theta(n)$)
 - Distribute the n input values into the buckets
 - Sort each bucket (insertion sort as default).
 - Go through the buckets in order, listing elements in each one
- **Input:** $A[1 \dots n]$, where $0 \leq A[i] < 1$ for all i
- **Output:** elements $A[i]$ sorted

Bucket Sort

Step 1: Distribute into Buckets



Step 2: Sort within each Bucket

Step 3: Concatenate all Buckets



Bucket Sort

BUCKET-SORT(A, n)

for $i \leftarrow 0$ **to** $n-1$

 insert $A[i]$ into list $B[\lfloor n A[i] \rfloor]$

for $i \leftarrow 0$ **to** $n-1$

 sort list $B[i]$ with insertion sort

concatenate the lists $B[0], B[1], \dots, B[n-1]$ in order

} $O(n)$

 $O(n_i^2)$

 $O(n)$

Where n_i is the size of bucket $B[i]$.

$$\begin{aligned} \text{Thus } T(n) &= \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \\ &= \Theta(n) + O(n) = \Theta(n) \end{aligned}$$

Summary: Sorting Algorithms

Insertion Sort: Suitable only for small $n \leq 50$ or nearly sorted inputs

Merge Sort: Guaranteed to be fast even in its worst case; stable

Quick Sort: Most useful general-purpose sorting for very little memory requirement and fastest average time. (Choose the median of three elements as pivot in practice)

Heap Sort: Requiring minimum memory and guaranteed to run fast; average and maximum time both roughly *twice* the average time of quick sort. Useful for time-critical applications

Counting Sort: Very useful when the keys have small range; stable; memory space for counters and for $2n$ records.

Radix Sort: Appropriate for keys either rather short or with an lexicographic collating sequence.

Bucket Sort: Assuming keys to have uniform distribution.

Complexity

Algorithm	Time Complexity			Worst Case Space Complexity
	Best	Average	Worst	
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Bucket Sort*	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$
Radix Sort**	$O(n * k)$	$O(n * k)$	$O(n * k)$	$O(n+k)$
Counting Sort***	$O(n+k)$	$O(n + k)$	$O(n+k)$	$O(k)$

*k represents the number of buckets

**k represents the number of digits in the largest number in the array

***k represents the difference between the smallest and the largest array value